Universidad Corporativa Coppel | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad | #1 | Estructuras de Control.

Lenguaje de Programación IV.

Ingeniería en Desarrollo de Software

academiaglobal

TUTOR: Aaron Iván Salazar Macías.

ALUMNO: Manuel Marin Sanchez.

FECHA: 19/03/2026.

Indicé.

Portada …… 1
Indicé ……... 2
Introducción …… 3
Investigación …….. 4
Desarrollo ……. 5,6,7
Conclusión …….. 8
Referencias bibliográficas …… 9

Introducción.

El Índice de Masa Corporal (IMC) se ha consolidado a nivel mundial como una herramienta fundamental en el ámbito de la salud pública para evaluar de manera preliminar el estado nutricional de las personas. Este indicador, que relaciona el peso y la estatura de un individuo, permite clasificar a la población en diferentes categorías que van desde el bajo peso hasta diversos grados de obesidad. En el contexto mexicano, esta medición adquiere una relevancia particularmente crítica, dado que nuestro país enfrenta una de las tasas más elevadas de sobrepeso y obesidad a nivel global, lo que representa un desafío significativo para el sistema de salud y demanda soluciones tecnológicas que auxilien al personal médico en sus labores cotidianas.

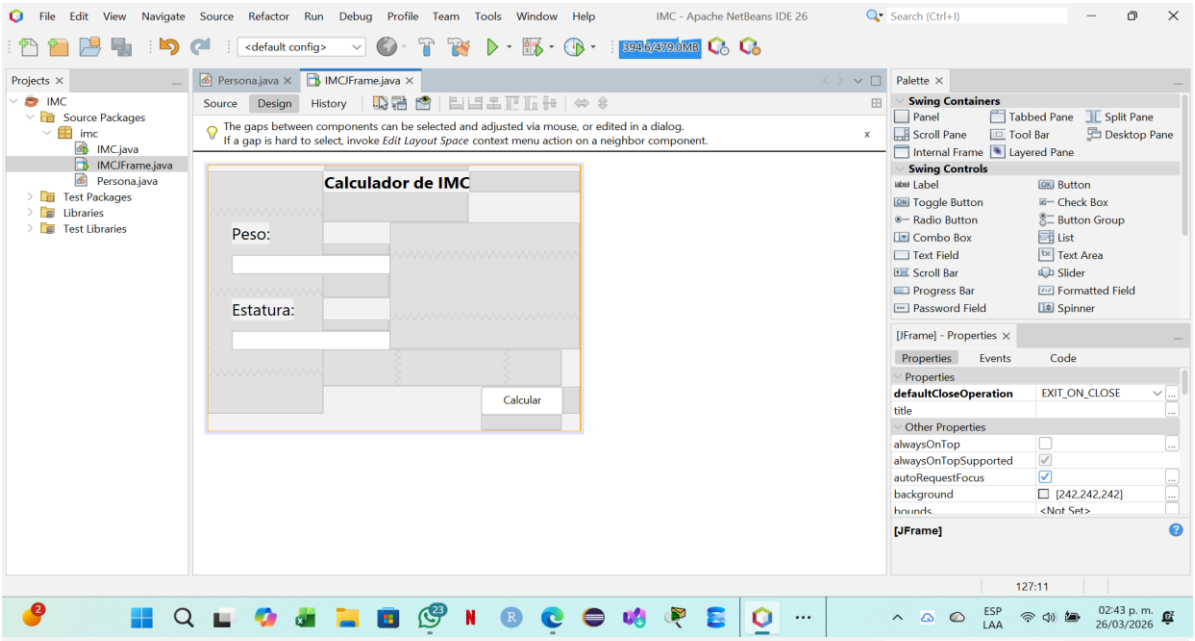
La presente actividad académica tiene como propósito fundamental desarrollar un sistema computacional en el lenguaje de programación Java 8, específicamente diseñado para calcular el IMC de los pacientes del Hospital de la Ciudad de México. El programa deberá solicitar al usuario dos datos esenciales: el peso expresado en kilogramos y la estatura expresada en metros. Una vez ingresados estos valores y realizada la operación matemática correspondiente (IMC = peso / estatura^2), el sistema mostrará no solo el resultado numérico obtenido, sino también la clasificación correspondiente según los estándares internacionales: bajo peso, peso normal, sobrepeso, obesidad grado I, obesidad grado II u obesidad grado III.

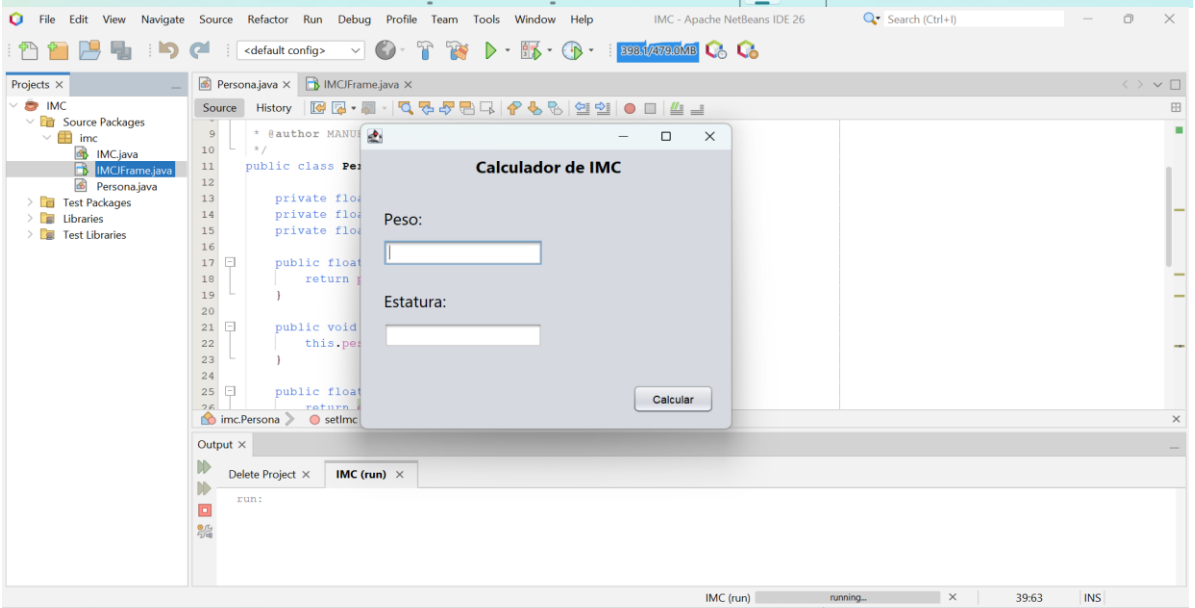
Resulta importante destacar que el desarrollo de esta aplicación deberá realizarse bajo los lineamientos del paradigma de Programación Orientada a Objetos (POO), haciendo uso de sus principios fundamentales y empleando estructuras de control como if-else, for o switch-case según lo requiera la lógica del programa. Además, se utilizará NetBeans como entorno de desarrollo integrado (IDE) y se implementará una interfaz gráfica amigable que permita una interacción intuitiva con el usuario final. El código fuente completo será alojado en la plataforma GitHub como parte de los entregables establecidos.

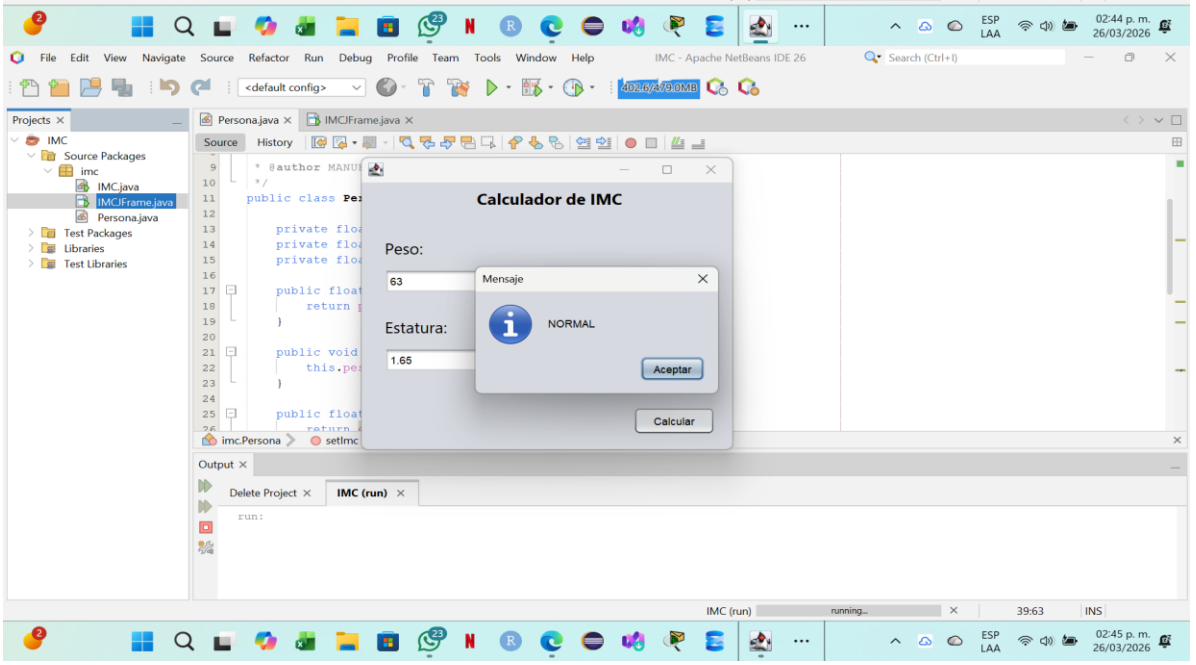
Investigación.

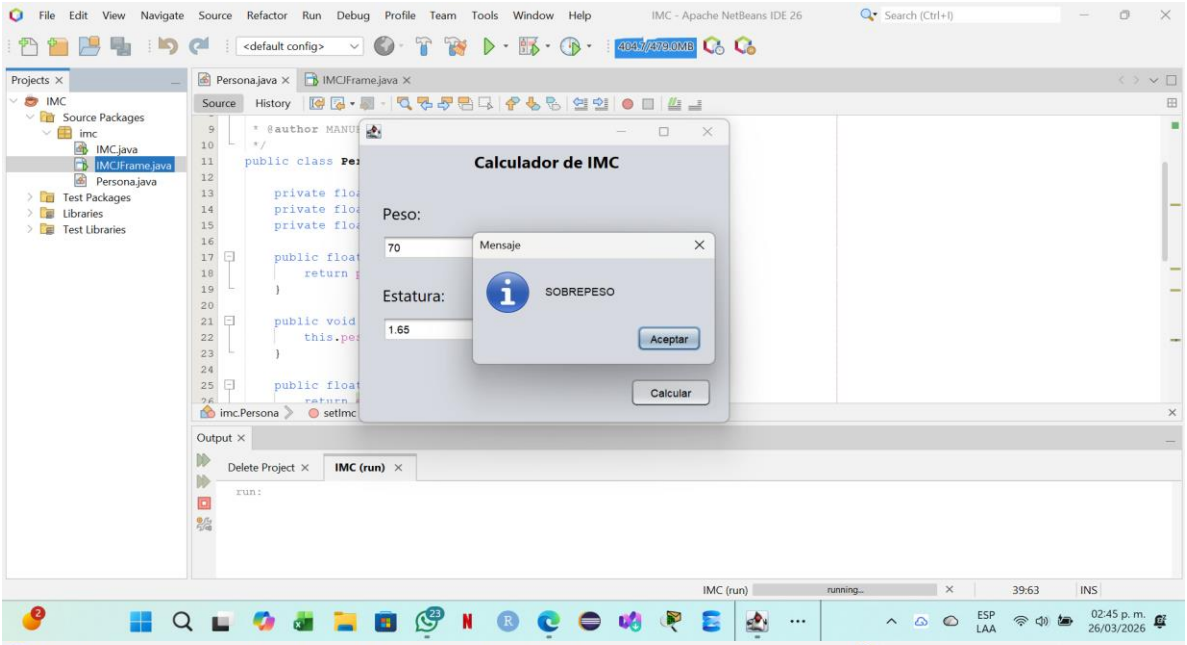
1. **Clase:** Es una plantilla o molde que define las características (atributos) y comportamientos (métodos) que tendrán los objetos de ese tipo. Es como un plano de construcción.
2. **Objeto:** Es una instancia de una clase. Es una entidad concreta que tiene estado (valores en sus atributos) y comportamiento (puede ejecutar métodos).
3. **Encapsulamiento:** Es el mecanismo que oculta los detalles internos de un objeto y protege sus datos. Se logra declarando atributos como private y proporcionando métodos públicos getter y setter para acceder a ellos.
4. **Herencia:** Permite crear una nueva clase basada en una clase existente. La clase hija (subclase) hereda los atributos y métodos de la clase padre (superclase), pudiendo agregar nuevos o modificar los existentes.
5. **Abstracción:** Consiste en simplificar la realidad modelando solo las características esenciales de un objeto, ignorando los detalles irrelevantes para el contexto del programa.
6. **Polimorfismo:** Capacidad de un objeto para comportarse de diferentes formas. Un mismo método puede tener diferentes implementaciones en distintas clases.
7. **Constructores:** Son métodos especiales que se ejecutan automáticamente al crear un objeto (con new). Sirven para inicializar los atributos del objeto.
8. **Eventos:** Son acciones que ocurren en el programa (como hacer clic en un botón) y que pueden ser detectadas para ejecutar código en respuesta.
9. **Atributos y Propiedades:** Los atributos son las variables que definen el estado de un objeto (ej: peso, estatura). Las propiedades son la forma de acceder a ellos (getters/setters).
10. **Métodos:** Son las funciones que definen el comportamiento de un objeto (ej: calcularIMC(), clasificarIMC()).

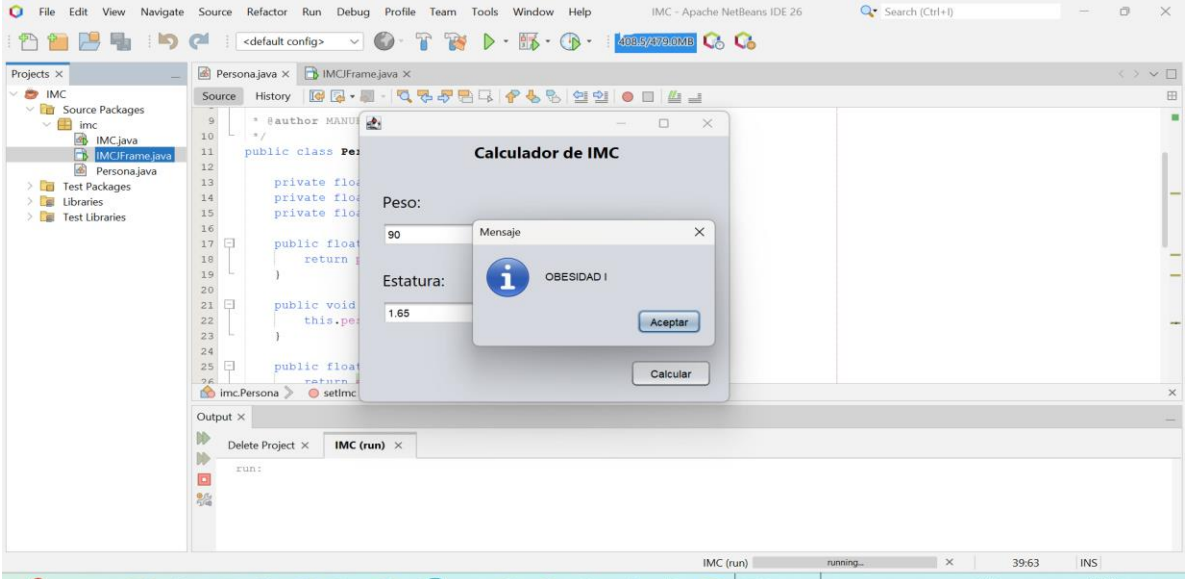
Desarrollo.

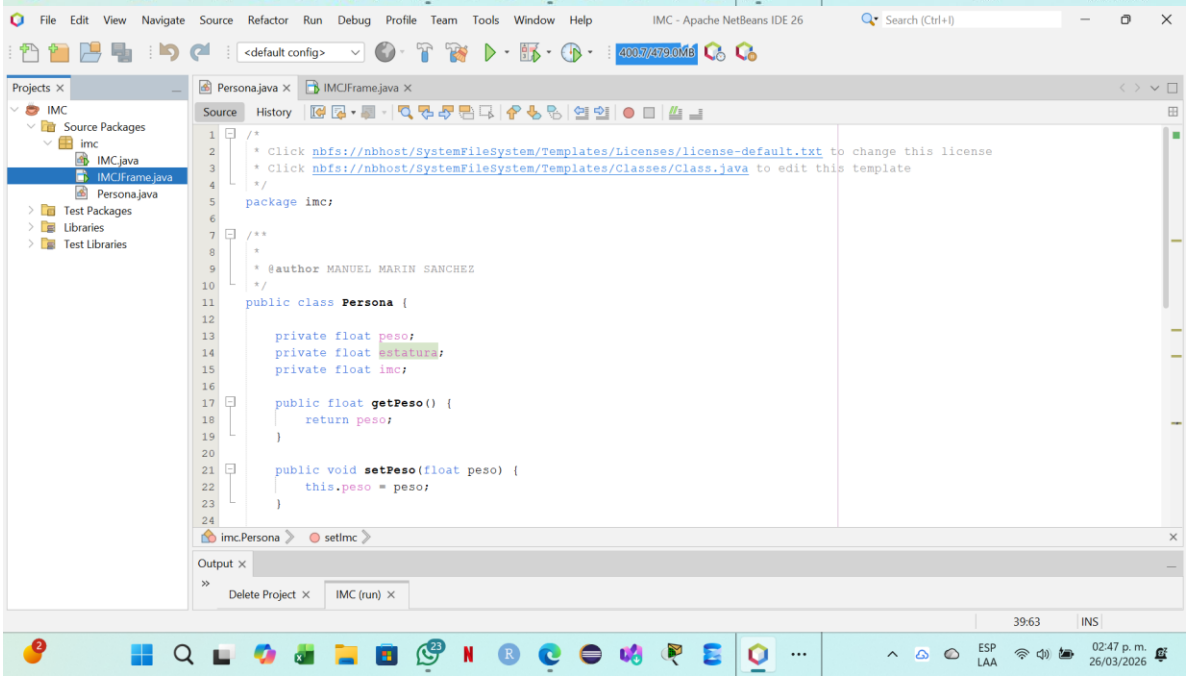

IMC - Apache NetBeans IDE 26
Calculador de IMC
Peso:
Estatura:
Calcular

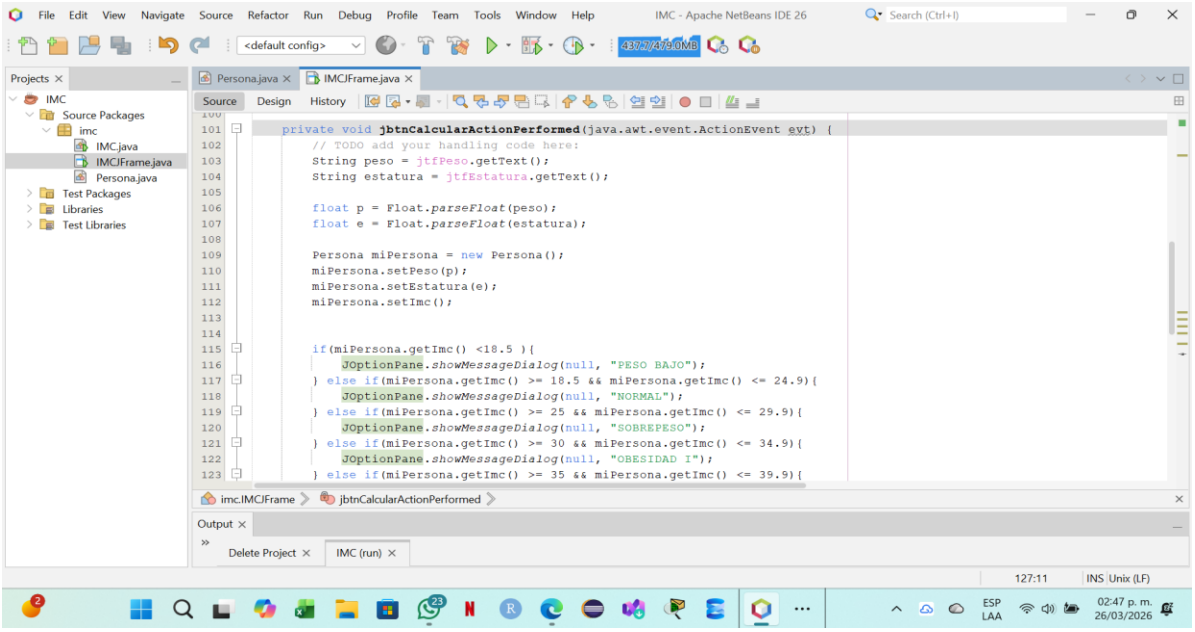

IMC - Apache NetBeans IDE 26
Calculador de IMC
Peso:
Estatura:
Calcular


IMC - Apache NetBeans IDE 26
Calculador de IMC
Peso:
63
Estatura:
1.65
Mensaje
NORMAL
Aceptar
Calcular


Calculador de IMC
Peso:
70
Estatura:
1.65
Mensaje
SOBREPESO
Aceptar
Calcular


Calculador de IMC
Peso:
90
Estatura:
1.65
Mensaje
OBESIDAD I
Aceptar
Calcular


/*
* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
* Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
*/
package imc;
/**
*
* @author MANUEL MARIN SANCHEZ
*/
public class Persona {
private float peso;
private float estatura;
private float imc;
public float getPeso() {
return peso;
}
public void setPeso(float peso) {
this.peso = peso;
}



Enlace a GitHub: https://github.com/ManuelMarin14/Manuel_Marin_Act2.git

Conclusión.

El desarrollo de la presente actividad ha permitido consolidar de manera significativa los conocimientos adquiridos a lo largo del curso de Lenguajes de Programación IV, particularmente en lo que respecta a la implementación práctica de los fundamentos de la Programación Orientada a Objetos (POO) en el lenguaje Java. La creación del sistema calculador de IMC para el Hospital de la Ciudad de México representó un ejercicio integral que demandó no solo habilidades técnicas de codificación, sino también una comprensión profunda de cómo estructurar adecuadamente un proyecto de software siguiendo las mejores prácticas profesionales.

Uno de los aprendizajes más valiosos obtenidos durante esta actividad fue la correcta aplicación del principio de encapsulamiento, el cual permitió proteger los datos de los pacientes mediante la declaración de atributos privados y el uso de métodos getters y setters para su manipulación controlada. Asimismo, la separación de responsabilidades en diferentes clases (Persona para representar al paciente, CalculadoraIMC para contener la lógica de negocio e IMCApp para gestionar la interfaz gráfica) demostró cómo la POO facilita la creación de código modular, reutilizable y fácil de mantener a largo plazo.

La implementación de la interfaz gráfica mediante Swing representó otro desafío importante que permitió comprender la diferencia entre la lógica de negocio y la capa de presentación, así como la importancia del manejo adecuado de eventos para lograr una experiencia de usuario fluida e intuitiva. Las validaciones implementadas para garantizar que el usuario ingrese únicamente valores numéricos positivos demostraron la relevancia de considerar los posibles errores humanos y proteger al programa de entradas no deseadas.

En conclusión, esta actividad no solo cumplió con el objetivo funcional de crear un sistema funcional para el cálculo del IMC, sino que también reforzó la importancia de aplicar rigurosamente los principios de la orientación a objetos en el desarrollo profesional de software. El código alojado en GitHub servirá como evidencia del trabajo realizado y como referencia para futuros proyectos.

Referencias.

1. Cosmina, L. (2018). *Java for Absolute Beginners: learn to program the fundamentals the Java 9+ ways*. Apress.
2. Deitel, P. J., & Deitel, H. M. (2016). *Java How to Program, Early Objects* (10th ed.). Pearson.
3. Friesen, J. (2019). *Java XML and JSON: document processing for Java SE*. Apress.
4. Oracle Corporation. (2023). *The Java™ Tutorials*. https://docs.oracle.com/javase/tutorial/
5. Schildt, H. (2018). *Java: The Complete Reference* (11th ed.). McGraw-Hill Education.